KubeCon CloudNativeCon North America 2026

Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena
AI Infrastructure Lead at Nunchux AI
CNCF TOC Alternate (GB)

Rainie Li
Engineering Manager at Pinterest
Apache YuniKorn PMC member



Speakers

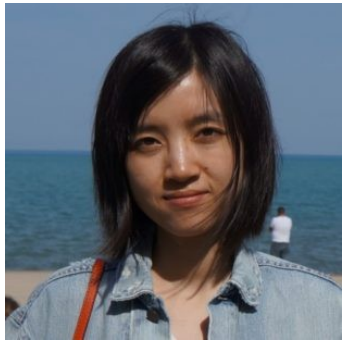


Ricardo Aravena

AI Infrastructure Lead, Nunchux AI

CNCF TOC Alternate (GB)
Cloud native and AI infrastructure

nunchux.ai



Rainie Li

Engineering Manager, Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

yunikorn.apache.org

KubeCon CloudNativeCon North America 2026

01 Unified scheduling

Different paths. Shared infrastructure.

01 Unified scheduling

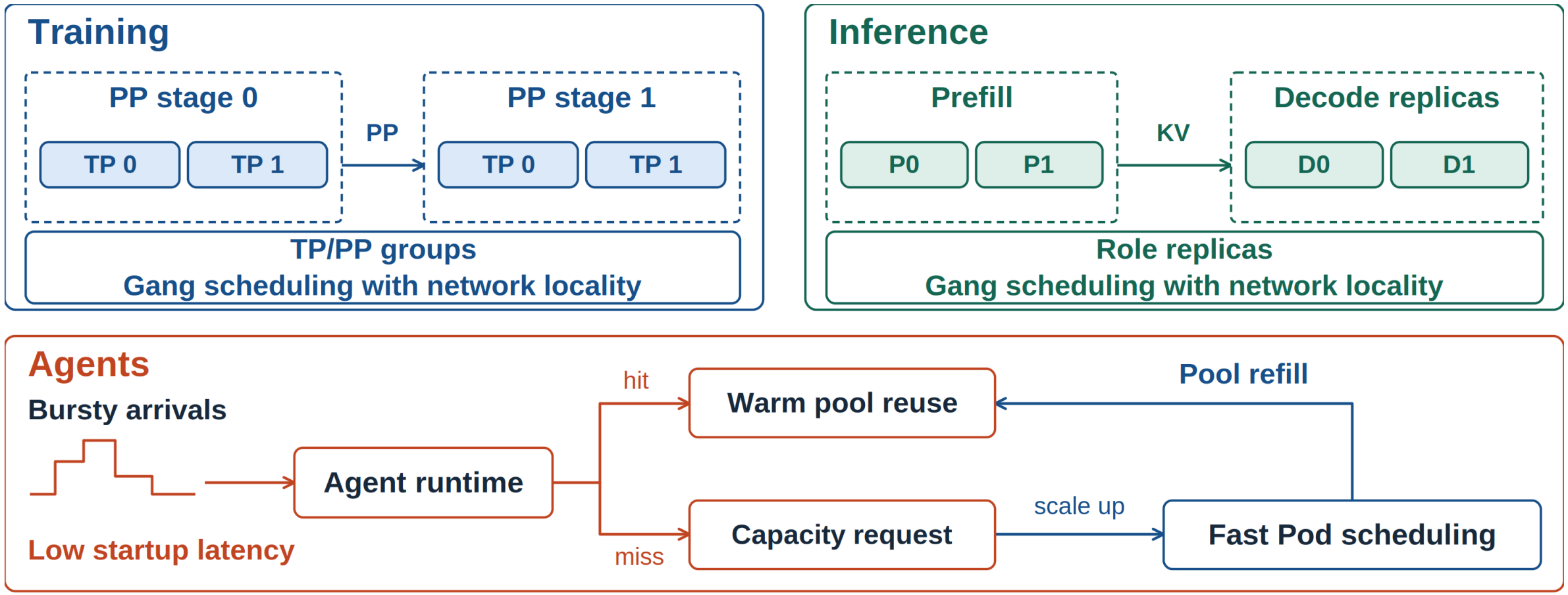
02 Multi-level gang & network topology

03 Ecosystem

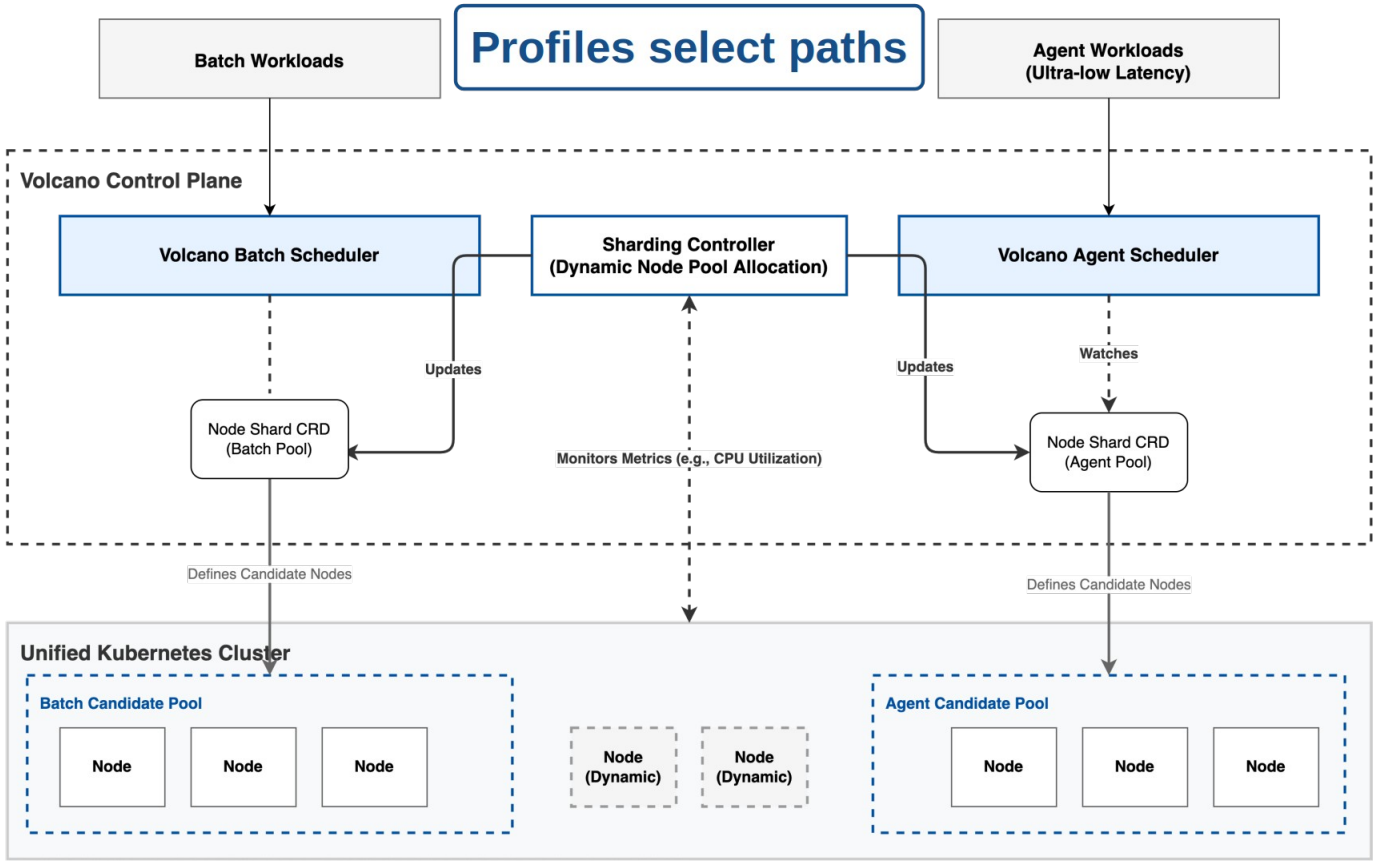
04 Benchmark framework & progress



Different Workloads, Shared Infrastructure



Unified Scheduling Framework



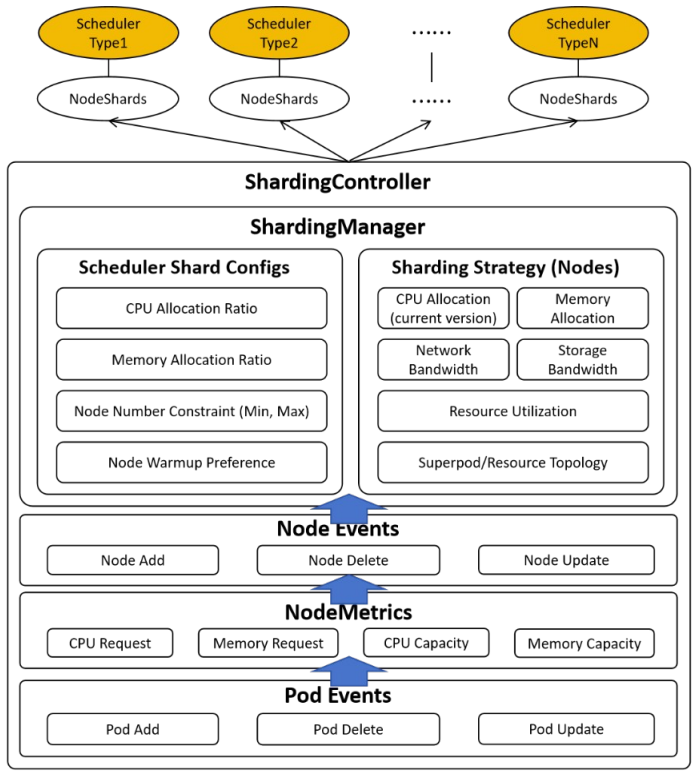
Sharding Controller

Live node state
Dynamic pool allocation

NodeShard

Candidate nodes
for each scheduling path

Dynamic Node Sharding



Node metrics + strategy
Compute desired node allocation

NodeShard spec and status

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Strategy result
Desired nodes

Scheduler state
Nodes in use

KubeCon CloudNativeCon
North America 2026

02 Multi-level gang & network topology

From available nodes to runnable workloads.

01 Unified scheduling

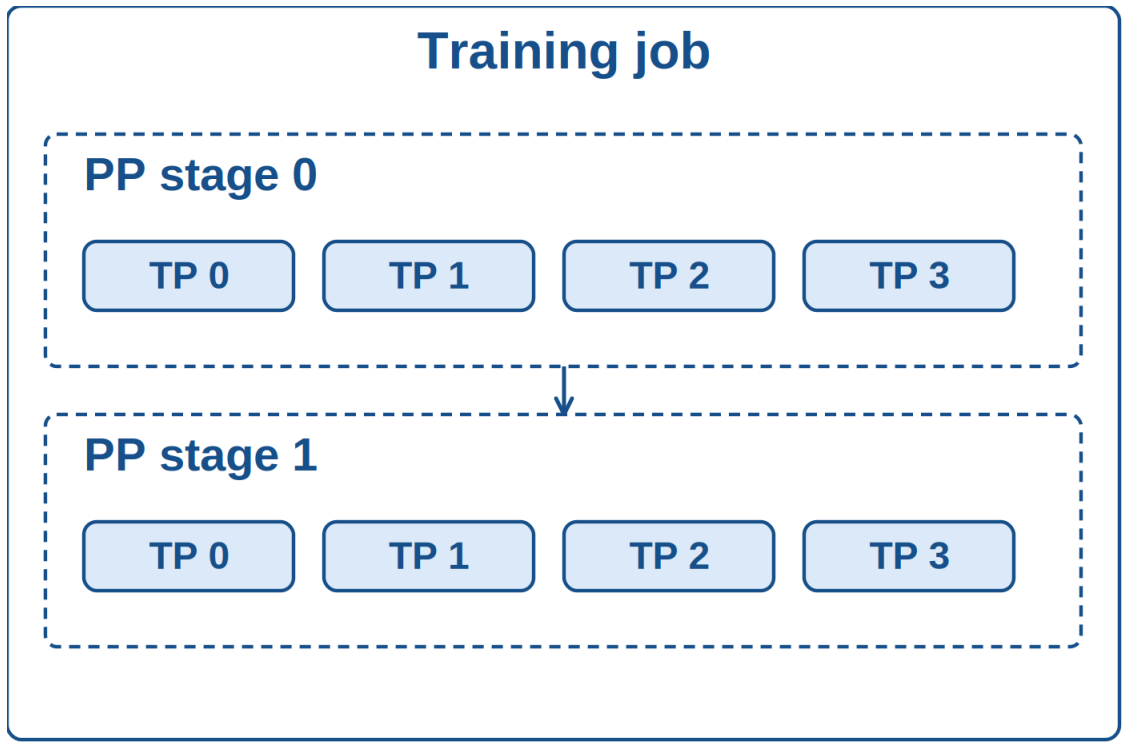
02 Multi-level gang & network topology

03 Ecosystem

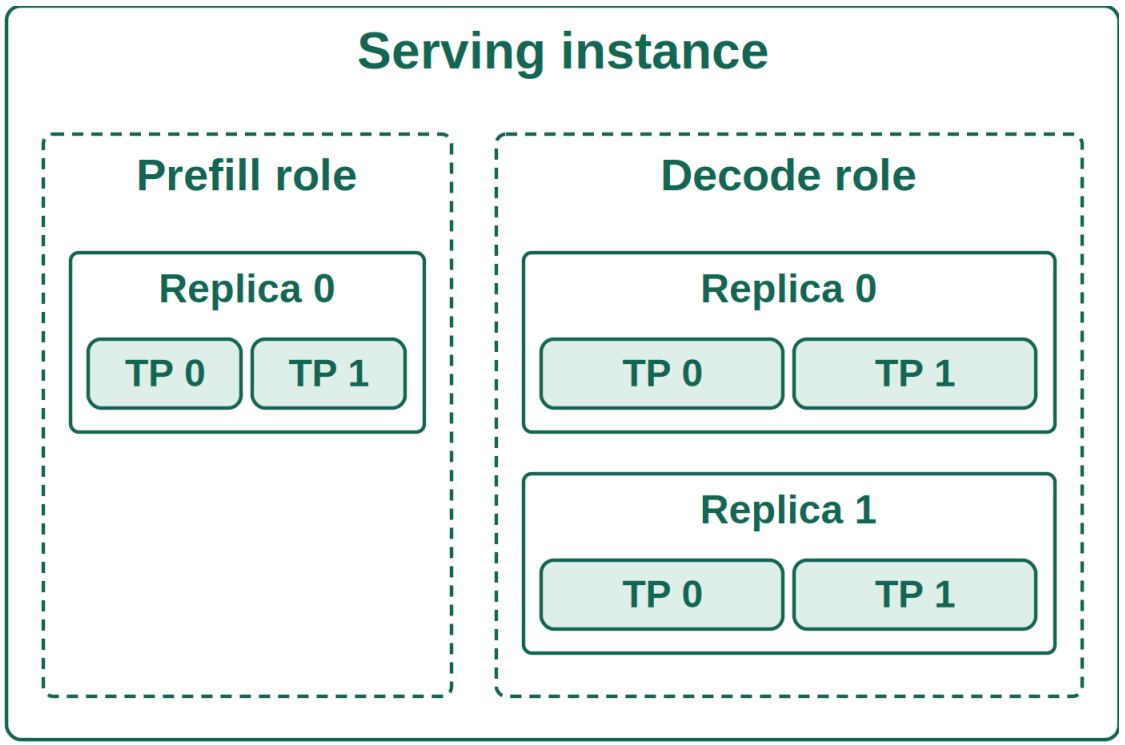
04 Benchmark framework & progress



Training and Inference Groups

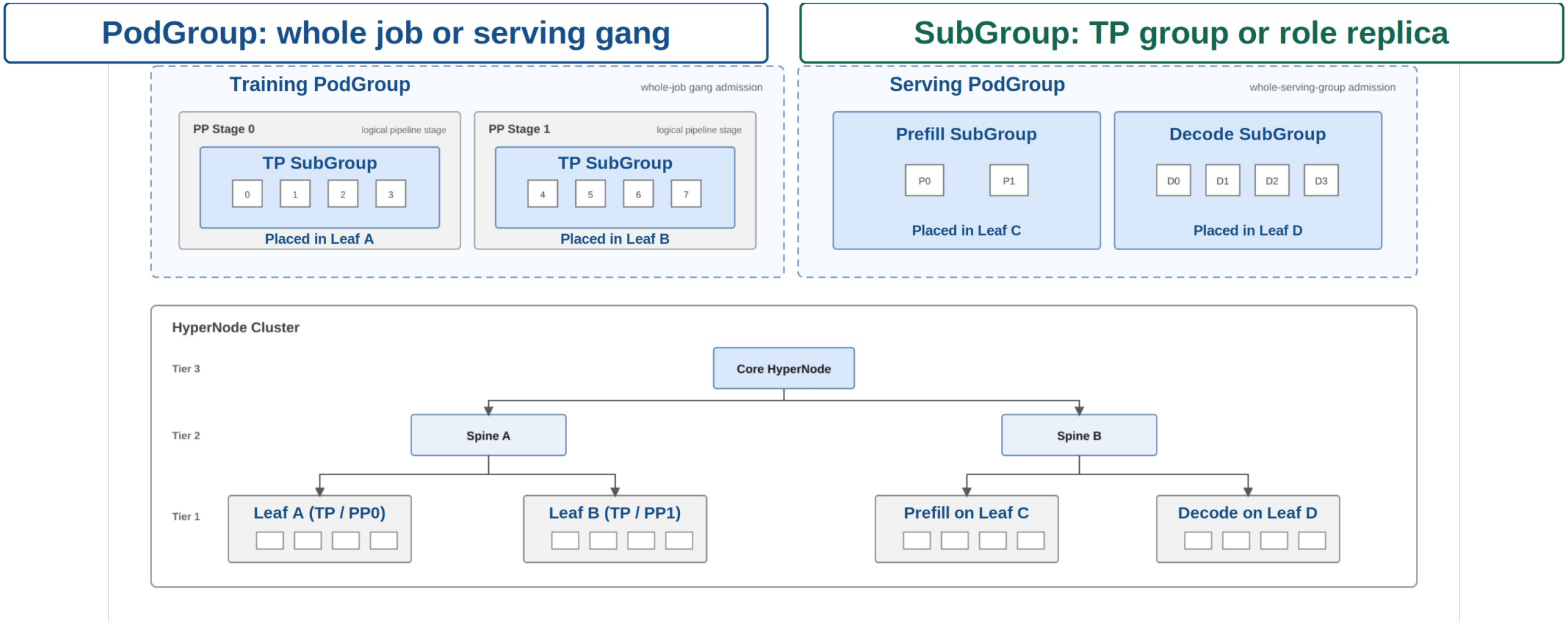


Complete TP groups within a job

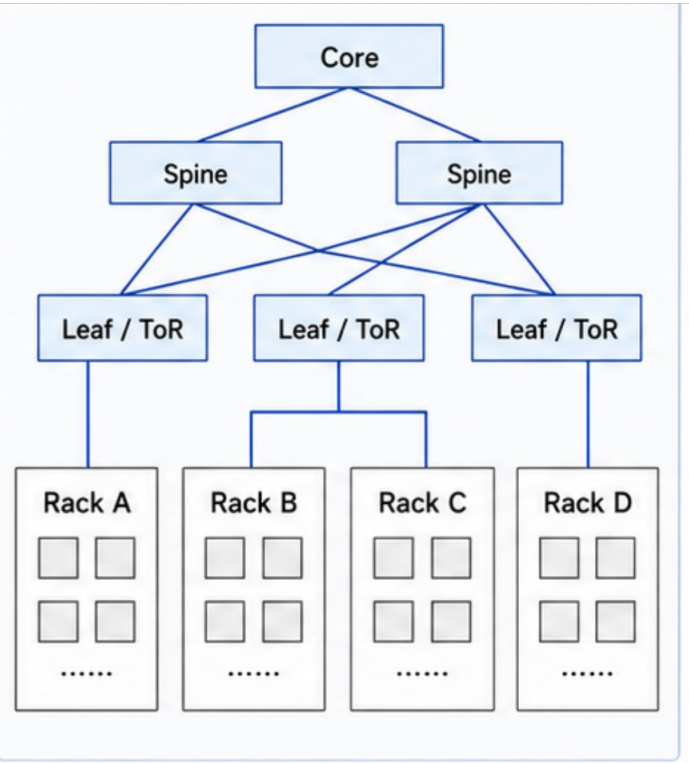


Complete replicas within a serving gang

Multi-Level Gang Scheduling



HyperNode: Network Topology



Parent HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Groups child HyperNodes

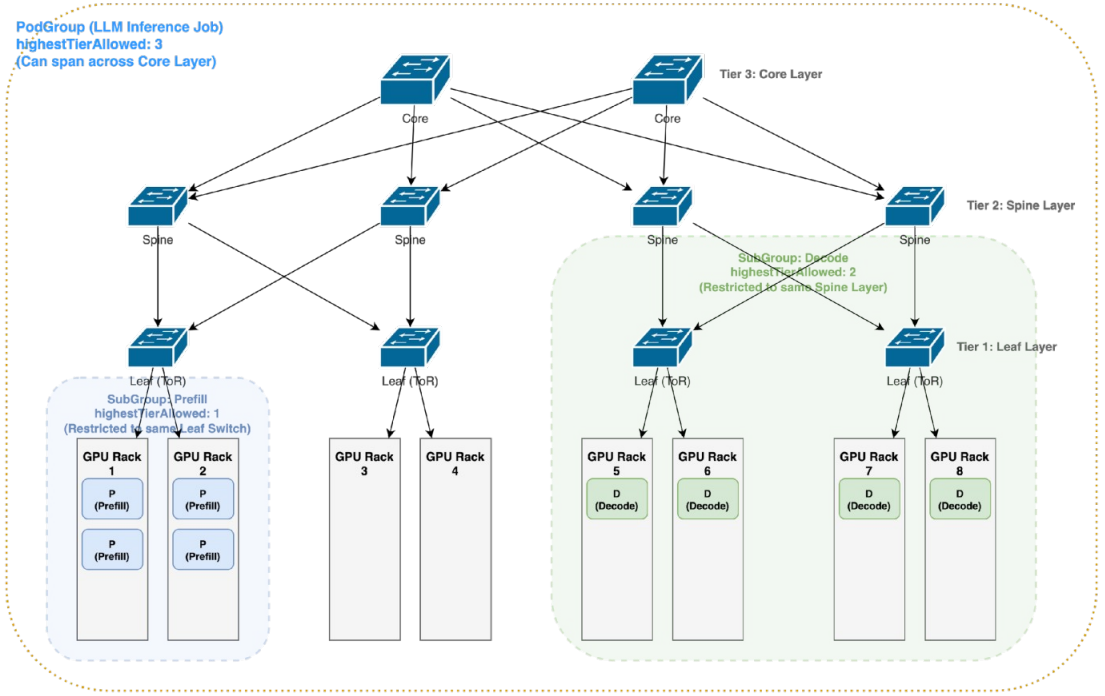
Leaf HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

Groups Kubernetes nodes

Place Groups at Different Topology Levels

Different SubGroups, different topology constraints



Prefill at leaf tier

Decode at spine tier

PodGroup.spec (example)

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    highestTierName: spine
```

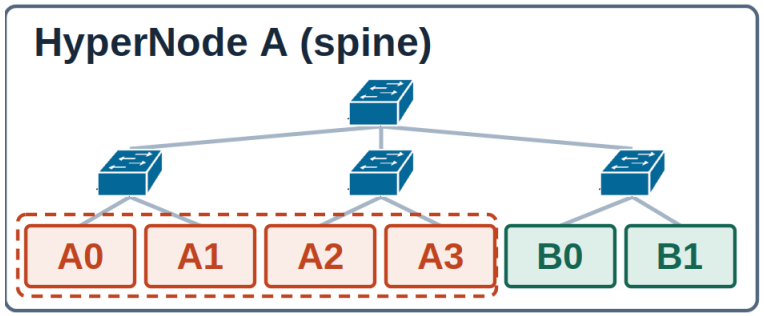
Gang-Aware Preemption

Incoming gang S
`minMember: 4`

S0 S1 S2 S3

Four Pods in one spine HyperNode
`highestTierName: spine`

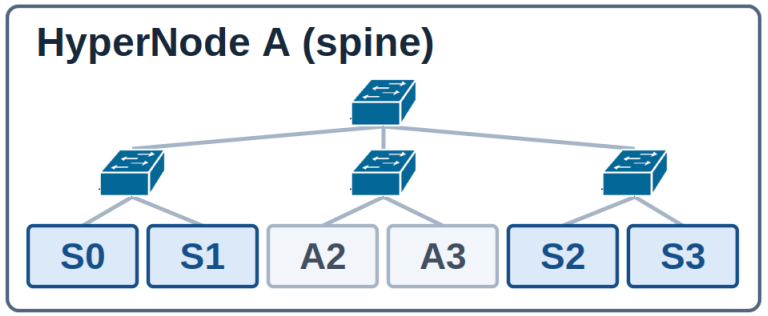
Before



Two complete training gangs

A: 4 Pods B: 4 Pods

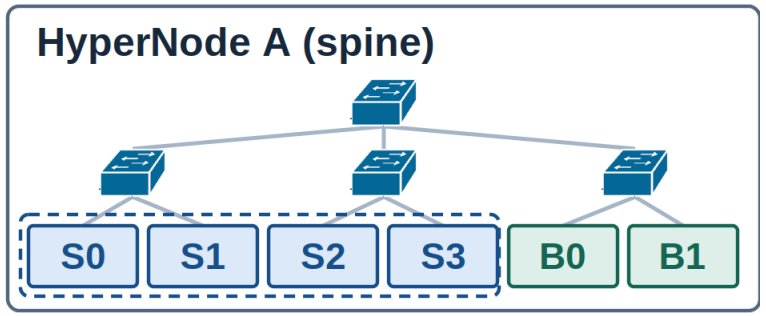
Pod-level victims



Two training jobs interrupted

Evict 2 Pods from A and 2 from B

Gang-aware victims



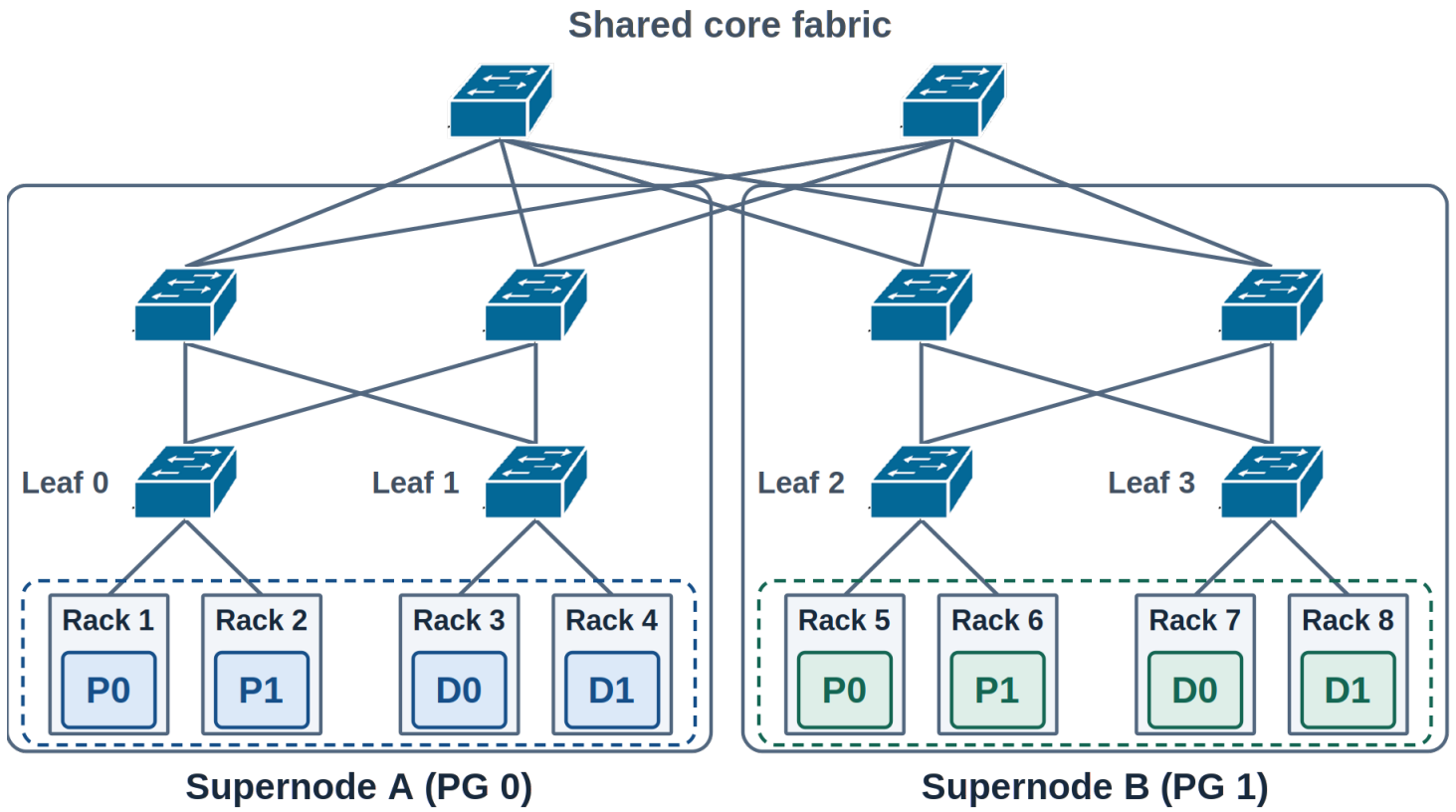
Training gang B stays complete

Evict the complete 4-Pod gang A

PodGroup Anti-Affinity

KubeCon CloudNativeCon North America 2026

Separate matching PodGroups across supernodes



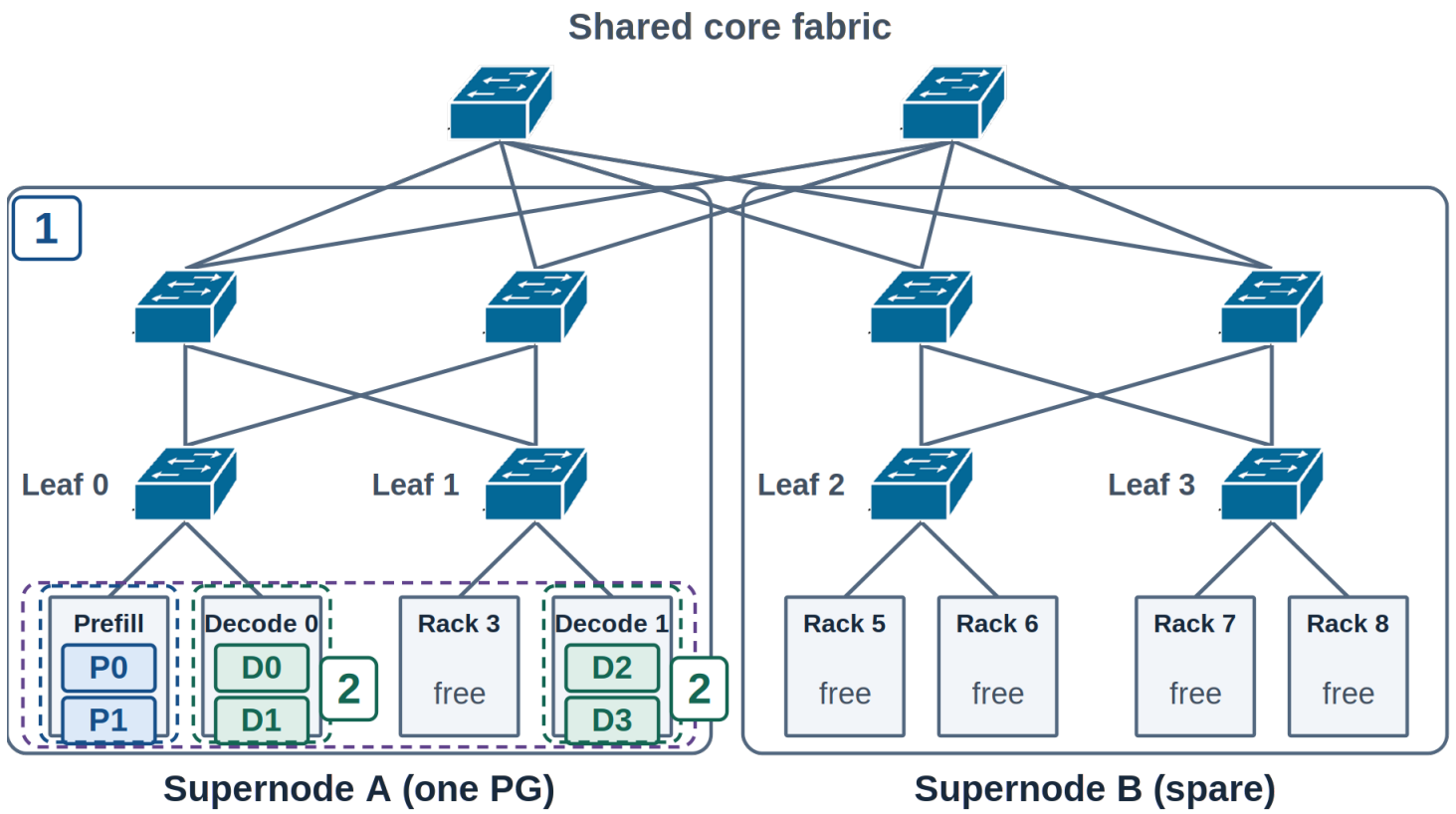
PodGroup.spec (key fields)

```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

PG label: app=model-a

Separate supernodes with PodGroup anti-affinity

SubGroup Affinity and Anti-Affinity



Planned

1 Affinity within a supernode

Prefill + Decode

2 Anti-affinity across leaves

Decode replica 0 / replica 1

KubeCon CloudNativeCon North America 2026

03 Ecosystem

Workload controllers carry scheduling intent.

01 Unified scheduling

02 Multi-level gang & network topology

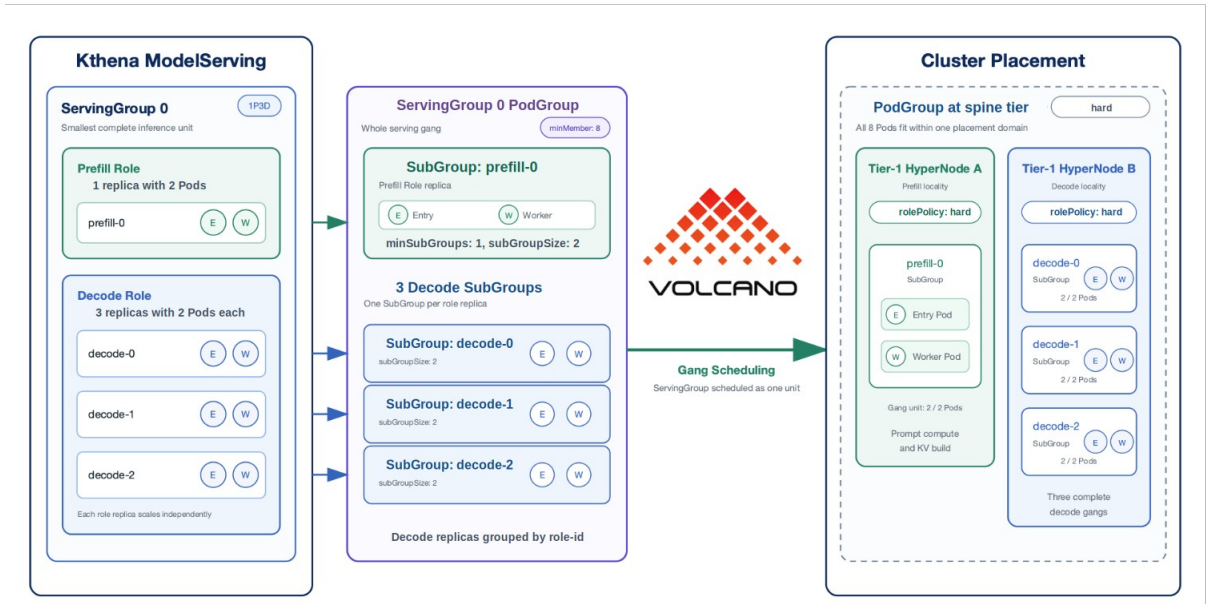
03 Ecosystem

04 Benchmark framework & progress



Kthena: Group and Role Topology

Kthena



Group: spine / Role replica: leaf

ModelServing.spec (excerpt)

```yaml
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  networkTopology:
    groupPolicy:
      mode: hard
      highestTierName: spine
    rolePolicy:
      mode: hard
      highestTierName: leaf
```

Kthena: Role Replica Mapping

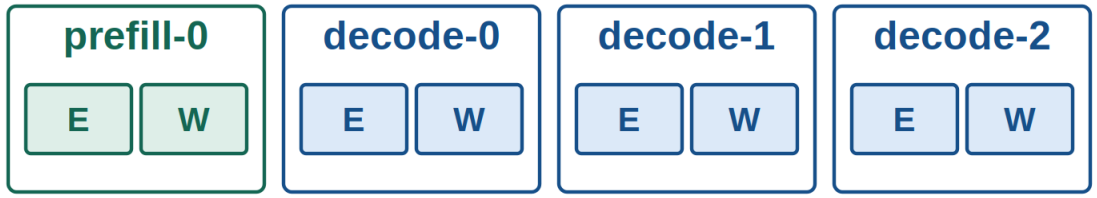
KubeCon CloudNativeCon North America 2026

Kthena

ModelServing.spec.template

```
roles:
- name: prefill
  replicas: 1
  workerReplicas: 1
- name: decode
  replicas: 3
  workerReplicas: 1
```

ServingGroup: 4 role replicas



Volcano

PodGroup.spec (role policies)

```
minMember: 8
subGroupPolicy:
- name: prefill
  minSubGroups: 1
  subGroupSize: 2
- name: decode
  minSubGroups: 3
  subGroupSize: 2
```

PodGroup: 4 SubGroups, 8 Pods



KubeRay: Multi-Host Replica Mapping

KubeCon CloudNativeCon North America 2026

KubeRay

RayCluster.spec

```
enableInTreeAutoscaling: true
workerGroupSpecs:
- groupName: worker
  minReplicas: 2
  replicas: 2
  maxReplicas: 4
  numOfHosts: 2
```

RayCluster: head and worker replicas



Volcano

PodGroup.spec (worker policy)

```
minMember: 5
subGroupPolicy:
- name: worker
  minSubGroups: 2
  subGroupSize: 2
  matchLabelKeys:
  - ray.io/worker-group-replica-name
```

PodGroup: 3 SubGroups, 5 Pods



Grove: PodGang Mapping

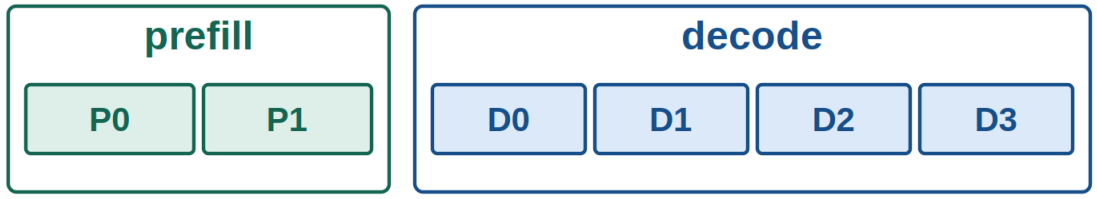
KubeCon CloudNativeCon North America 2026

Grove

PodGang.spec

```
podGroups:
- name: prefill
  minReplicas: 2
- name: decode
  minReplicas: 4
```

PodGang: 2 named groups



Volcano

PodGroup.spec (group policies)

```
minMember: 6
subGroupPolicy:
- name: prefill
  minSubGroups: 1
  subGroupSize: 2
- name: decode
  minSubGroups: 1
  subGroupSize: 4
```

PodGroup: 2 SubGroups, 6 Pods



Ecosystem Roadmap



RoleBasedGroup (RBG)

Integration in progress

RoleInstance gang mapping

HyperNode tree queries



Kubeflow

Planned

Training group structure

Network topology constraints



LeaderWorkerSet (LWS)

Planned

Multi-role and multi-group support

Relationships across groups

KubeCon CloudNativeCon North America 2026

04 Benchmark framework & progress

Measure behavior across workload shapes.

01 Unified scheduling

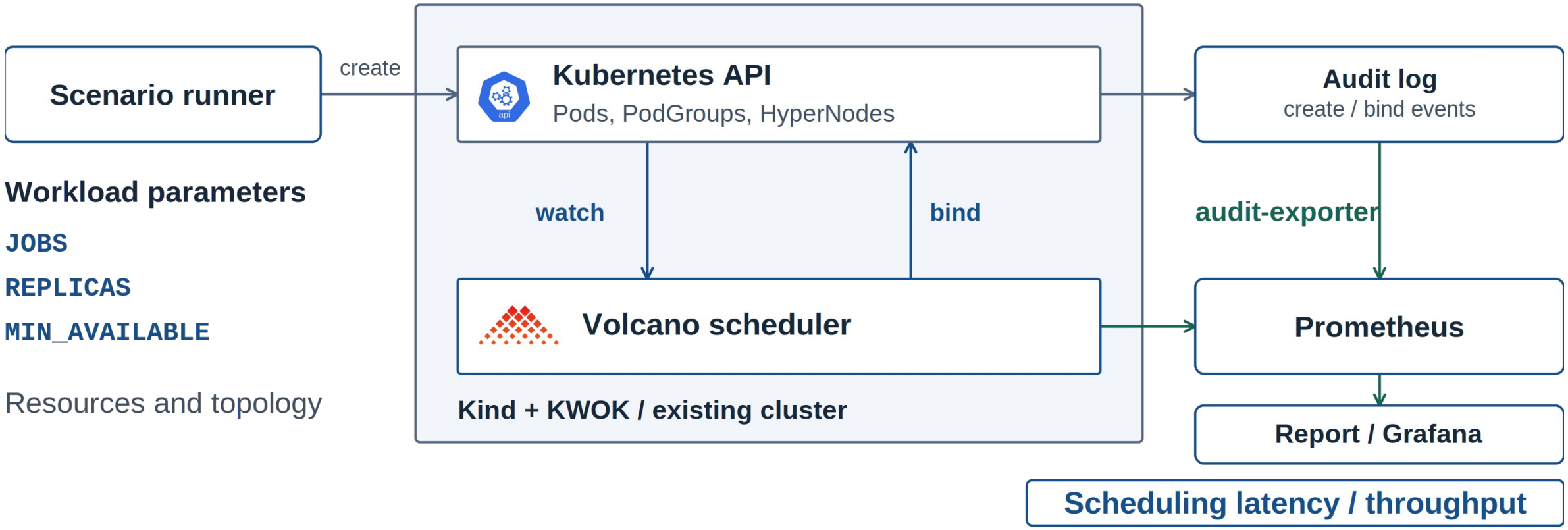
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress



Scheduling Benchmark Framework

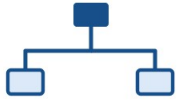


Cross-Scheduler Benchmarks

KubeCon CloudNativeCon North America 2026

Matched workloads and cluster capacity

Comparisons planned

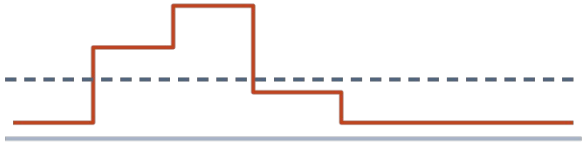
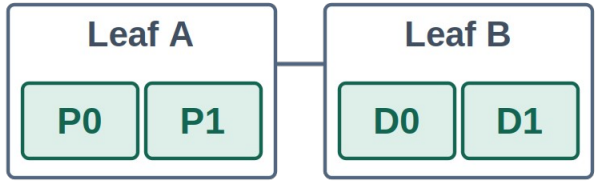
Scenario	Volcano	Kubernetes scheduler	Measurements
 **Bursty arrivals**	**Planned**	Planned	Tail latency Backlog recovery
 **Mixed workloads**	**Planned**	Planned	Binding throughput Wait time by workload
 **Complete gangs**	**Planned**	Capability check	Gang completion Partial placement
 **Network topology**	**Planned**	Capability check	Placement validity Gang admission latency

Find the scenarios each scheduler handles well

More schedulers to follow

Workload and Topology Test Matrix

KubeCon CloudNativeCon North America 2026

Scenario		Variables	Metrics
Bursts		**Arrival rate and capacity** Pool capacity requests	**Tail latency** Backlog recovery
Gang size		**Group size and fragmentation** Large / small gangs	**Complete gangs** Binding throughput
Topology		**Tier limits and domains** Gang with anti-affinity	**Valid placement** Gang latency

v1.17 Priorities

KubeCon CloudNativeCon North America 2026

Preemption	**Fewer unnecessary evictions**	Gang admission + victim selection Topology-aware reclaim under review
Group placement	**SubGroup affinity / anti-affinity**	Locality and replica isolation API and integration work planned
Performance	**Cache locks + queue status cost**	Replay workloads and profile bottlenecks Scheduler comparisons and topology tests next

Volcano Users



Thank you

Questions & contributions

volcano.sh

GitHub

Slack